

```

1  /-!
2  # Exotic matter, in Lean 4
3
4  A diagonal ("Hawking-Ellis type I") stress-energy tensor, seen in a local orthonormal
5  frame, is fixed by an energy density ` $\rho$ ` and three principal pressures ` $p_1 p_2 p_3$ `.
6  Ordinary matter obeys the classical *energy conditions*; exotic matter breaks at least one.
7
8      DEC  $\longrightarrow$  WEC  $\longrightarrow$  NEC  $\longleftarrow$  SEC      (an arrow means "implies")
9
10 Scalars are `Int` (multiples of some fixed unit), so this file needs only core Lean 4.
11 Every proof is plain ordered arithmetic, so it ports to ` $\mathbb{R}$ ` with Mathlib by swapping
12 ` $\omega$ ` for `linarith`.
13 -/
14
15 /-- Stress-energy of a diagonal fluid: energy density ` $\rho$ ` and principal pressures ` $p_i$ `. -/
16 structure Fluid where
17    $\rho$  : Int
18    $p_1$  : Int
19    $p_2$  : Int
20    $p_3$  : Int
21   deriving Repr, DecidableEq
22
23 namespace Fluid
24
25 /-- Null energy condition: ` $\rho + p_i \geq 0$ `. -/
26 def NEC (T : Fluid) : Prop :=  $0 \leq T.\rho + T.p_1 \wedge 0 \leq T.\rho + T.p_2 \wedge 0 \leq T.\rho + T.p_3$ 
27
28 /-- Weak energy condition: every observer measures ` $\rho \geq 0$ `. -/
29 def WEC (T : Fluid) : Prop :=  $0 \leq T.\rho \wedge T.NEC$ 
30
31 /-- Strong energy condition: gravity attracts, ` $\rho + \sum p_i \geq 0$ `. -/
32 def SEC (T : Fluid) : Prop :=  $0 \leq T.\rho + T.p_1 + T.p_2 + T.p_3 \wedge T.NEC$ 
33
34 /-- Dominant energy condition: ` $\rho \geq |p_i|$ `, so energy never flows faster than light. -/
35 def DEC (T : Fluid) : Prop :=
36    $(-T.\rho \leq T.p_1 \wedge T.p_1 \leq T.\rho) \wedge (-T.\rho \leq T.p_2 \wedge T.p_2 \leq T.\rho) \wedge (-T.\rho \leq T.p_3 \wedge T.p_3 \leq T.\rho)$ 
37
38 /-- Ordinary matter obeys every energy condition. -/
39 def Ordinary (T : Fluid) : Prop := T.NEC  $\wedge$  T.WEC  $\wedge$  T.SEC  $\wedge$  T.DEC
40
41 /-- **Exotic matter** breaks at least one energy condition. -/
42 def Exotic (T : Fluid) : Prop :=  $\neg$  T.Ordinary
43

```

```

44 /-- The really exotic stuff (traversable wormholes, warp drives): breaks the null condition. -/
45 def NullExotic (T : Fluid) : Prop := ¬ T.NEC
46
47 instance (T : Fluid) : Decidable T.NEC := inferInstanceAs (Decidable ( _ ∧ _ ∧ _ ))
48 instance (T : Fluid) : Decidable T.WEC := inferInstanceAs (Decidable ( _ ∧ _ ))
49 instance (T : Fluid) : Decidable T.SEC := inferInstanceAs (Decidable ( _ ∧ _ ))
50 instance (T : Fluid) : Decidable T.DEC := inferInstanceAs (Decidable (( _ ∧ _ ) ∧ ( _ ∧ _ ) ∧ ( _ ∧
  _ )))
51 instance (T : Fluid) : Decidable T.Ordinary := inferInstanceAs (Decidable ( _ ∧ _ ∧ _ ∧ _ ))
52 instance (T : Fluid) : Decidable T.Exotic := inferInstanceAs (Decidable (¬ _ ))
53 instance (T : Fluid) : Decidable T.NullExotic := inferInstanceAs (Decidable (¬ _ ))
54
55 /-! ## The hierarchy of conditions -/
56
57 theorem WEC.nec {T : Fluid} (h : T.WEC) : T.NEC := h.2
58 theorem SEC.nec {T : Fluid} (h : T.SEC) : T.NEC := h.2
59
60 theorem DEC.wec {T : Fluid} (h : T.DEC) : T.WEC := by
61   unfold DEC WEC NEC at *; omega
62
63 /-- Ordinary matter is exactly matter obeying the strong and dominant conditions. -/
64 theorem ordinary_iff (T : Fluid) : T.Ordinary ↔ T.SEC ∧ T.DEC := by
65   unfold Ordinary SEC DEC WEC NEC; omega
66
67 /-- Breaking the null condition breaks the weak one too. -/
68 theorem NullExotic.not_wec {T : Fluid} (h : T.NullExotic) : ¬ T.WEC := fun hw => h hw.nec
69
70 theorem NullExotic.exotic {T : Fluid} (h : T.NullExotic) : T.Exotic := fun ho => h ho.1
71
72 /-- Negative energy density is exotic. -/
73 theorem exotic_of_neg_energy {T : Fluid} (h : T.p < 0) : T.Exotic := by
74   unfold Exotic Ordinary NEC WEC SEC DEC; omega
75
76 /-! ## A zoo of fluids -/
77
78 /-- Pressureless dust. -/
79 def dust : Fluid := ⟨1, 0, 0, 0⟩
80 /-- Radiation, `p = ρ/3`. -/
81 def radiation : Fluid := ⟨3, 1, 1, 1⟩
82 /-- A cosmological constant `Λ`: `p = -ρ`. Positive `Λ` is dark energy. -/
83 def cosmoConst (Λ : Int) : Fluid := ⟨Λ, -Λ, -Λ, -Λ⟩
84 /-- Anti-de Sitter vacuum: negative cosmological constant. -/
85 def adsVacuum : Fluid := cosmoConst (-1)

```

```

86 /-- Phantom energy, `p = wp` with `w = -3/2 < -1`. -/
87 def phantom : Fluid := ⟨2, -3, -3, -3⟩
88 /-- Casimir vacuum between parallel plates: `ε · diag(-1, 1, 1, -3)` with `ε = π²ħc / (720 a⁴)`.
89 -/
89 def casimir (ε : Int) : Fluid := ⟨-ε, ε, ε, -3 * ε⟩
90 /-- Morris-Thorne wormhole throat (`b' = 1/2`): radial tension `4` beats density `2`. -/
91 def wormholeThroat : Fluid := ⟨2, -4, 1, 1⟩
92
93 /-- Dark energy is only *mildly* exotic: it never breaks NEC... -/
94 theorem cosmoConst_nec (Λ : Int) : (cosmoConst Λ).NEC := by
95   simp only [cosmoConst, NEC]; omega
96
97 /-- ...it just stops gravity from attracting. -/
98 theorem cosmoConst_not_sec {Λ : Int} (h : 0 < Λ) : ¬ (cosmoConst Λ).SEC := by
99   simp only [cosmoConst, SEC, NEC]; omega
100
101 /-- The Casimir vacuum breaks the null condition for *every* plate separation. -/
102 theorem casimir_nullExotic {ε : Int} (h : 0 < ε) : (casimir ε).NullExotic := by
103   simp only [casimir, NullExotic, NEC]; omega
104
105 /-- Morris-Thorne flare-out: radial tension `τ` exceeding the density `ρ` breaks NEC. -/
106 theorem throat_nullExotic {ρ τ pt : Int} (h : ρ < τ) : (Fluid.mk ρ (-τ) pt pt).NullExotic := by
107   simp only [NullExotic, NEC]; omega
108
109 example : dust.Ordinary := by decide
110 example : radiation.Ordinary := by decide
111 example : (cosmoConst 1).NEC ∧ (cosmoConst 1).WEC ∧ (cosmoConst 1).DEC := by decide
112 example : adsVacuum.NEC ∧ ¬ adsVacuum.WEC := by decide
113 example : phantom.NullExotic := by decide
114 example : wormholeThroat.NullExotic := by decide
115
116 /-- First (strongest) energy condition a fluid violates. -/
117 def classify (T : Fluid) : String :=
118   if ¬ T.NEC then "exotic: breaks NEC (wormhole / warp-drive grade)"
119   else if ¬ T.WEC then "exotic: negative energy density"
120   else if ¬ T.SEC then "exotic: gravitationally repulsive"
121   else if ¬ T.DEC then "exotic: superluminal energy flow"
122   else "ordinary"
123
124 end Fluid
125
126 /-! ## Exotic matter as a type -/
127

```

```

128 open Fluid
129
130 /-- Exotic matter: a fluid bundled with a proof that it breaks an energy condition. -/
131 abbrev ExoticMatter := {T : Fluid // T.Exotic}
132
133 /-- Here is one: a Casimir vacuum, certified exotic. -/
134 def casimirVacuum : ExoticMatter := ⟨casimir 1, by decide⟩
135
136 /-- And a phantom fluid. -/
137 def phantomFluid : ExoticMatter := ⟨phantom, by decide⟩
138
139 /-- Exotic matter in the sci-fi sense: Newton's  $m \cdot a = F$  with  $m < 0$  accelerates against the
140 force. -/
141 theorem negative_mass_recoils {m a F : Int} (hm : m < 0) (h : m * a = F) (hF : 0 < F) : a < 0 :=
142   by
143     by_cases ha : a < 0
144     · exact ha
145     · have h1 : 0 ≤ (-m) * a := Int.mul_nonneg (by omega) (by omega)
146       rw [Int.neg_mul] at h1
147       omega
148
149 #eval do
150   for (name, T) in [("dust", dust), ("radiation", radiation), ("dark energy", cosmoConst 1),
151     ("AdS vacuum", adsVacuum), ("phantom energy", phantom), ("Casimir vacuum", casimir 1),
152     ("wormhole throat", wormholeThroat)] do
153     IO.println s!"{name}: {T.classify}"

```